

Semana 5 — Containerização e CI/CD

Documentação técnica do desafio Do Dev ao Deploy

1. Identificação

Integrantes: João Pedro Pellegrini

Repositório: https://github.com/jppellearch/Projeto_Semana_5

Descrição: Aplicação com backend em Django, frontend em Next.js, banco PostgreSQL e Nginx como reverse proxy. O objetivo do desafio foi containerizar toda a aplicação, montar um pipeline de CI/CD e publicar as imagens de produção no GHCR.

2. Arquitetura

Stack: Django + Django REST Framework (backend), Next.js com App Router (frontend), PostgreSQL 16 (banco) e Nginx (reverse proxy e SSL).

Serviços: db, backend, frontend e nginx, cada um em seu próprio container. Em produção, só o Nginx fica exposto ao host (portas 80 e 443); os outros três serviços só são acessíveis pela rede interna do Docker Compose.

Fluxo de comunicação: O cliente acessa o Nginx via HTTPS. As rotas /api/ e /admin/ são encaminhadas para o backend, e a rota / para o frontend. O backend se conecta ao banco pelo nome de serviço db, na porta 5432.

3. Etapa 1 – Containerização de DEV

Implementação: Criamos um Dockerfile para o backend (baseado em python:3.12-slim) e outro para o frontend (baseado em node:20-alpine), ambos voltados só para desenvolvimento. Usamos bind mounts para que qualquer alteração no código apareça na hora, sem precisar reconstruir a imagem.

Validação: Subimos cada container separadamente com docker run e confirmamos que o hot reload funcionava nos dois: uma mudança salva no editor já aparecia no navegador em segundos.

4. Etapa 2 – Docker Compose (DEV)

Implementação: Criamos um docker-compose.yml na raiz do projeto, unindo os três serviços (db, backend e frontend) para subir tudo com um único comando.

Healthcheck: Configuramos um healthcheck no banco com pg_isready, para o backend só iniciar depois que o PostgreSQL estivesse realmente pronto para aceitar conexões.

Persistência: Usamos um volume nomeado (postgres_data) para os dados do banco não se perderem toda vez que o container reiniciasse.

Validação: Rodamos docker compose up e confirmamos que a stack inteira subia sem erro, com o frontend conseguindo buscar os dados da API do backend normalmente.

5. Etapa 3 – Pipeline de CI

Jobs do backend: lint-backend (Flake8) → build-backend (build da imagem) → test-backend (testes do Django), cada um dependendo do anterior.

Jobs do frontend: lint-frontend (ESLint) → build-frontend (build da imagem) → test-frontend (Jest), na mesma lógica do backend.

Fail-Fast: Testamos de propósito três tipos de falha em cada trilha (erro de lint, erro de build e teste quebrado) e confirmamos que, assim que uma etapa falha, as seguintes nem chegam a rodar. Depois corrigimos cada erro e validamos que as duas trilhas voltavam a ficar verdes.

Cache: Configuramos cache de dependências (pip no backend, npm no frontend) para deixar as execuções seguintes do pipeline mais rápidas.

6. Etapa 4 – Containers de Produção

Backend: Reescrevemos o Dockerfile de produção usando gunicorn no lugar do servidor de desenvolvimento do Django.

Frontend: Ativamos output: standalone no Next.js, para empacotar só o necessário para rodar em produção.

Multi-stage: Nos dois casos usamos build em múltiplos estágios: um estágio compila tudo (inclusive as ferramentas pesadas), e o estágio final copia só o que já está pronto, sem levar compiladores nem código-fonte de build para a imagem final. Isso foi essencial no backend: numa primeira tentativa sem multi-stage, mesmo removendo os pacotes de compilação no fim do Dockerfile, a imagem continuou gigante (1,14 GB), porque cada instrução RUN vira uma camada própria e apagar arquivos depois não recupera o espaço das camadas anteriores. Migrando para multi-stage, o tamanho caiu bastante.

Usuários não-root: O backend roda como o usuário appuser e o frontend como nextjs — nenhum dos dois containers de produção usa root.

Tamanho final das imagens: backend: [PREENCHER — tamanho obtido] · frontend: [PREENCHER — tamanho obtido]

7. Etapa 5 – Nginx e SSL

Reverse proxy: O Nginx encaminha /api/ e /admin/ para o backend e / para o frontend, usando proxy_pass.

Portas expostas: Só o Nginx publica portas no host (80 e 443). Backend, frontend e banco ficam acessíveis apenas pela rede interna do Compose.

HTTPS: Geramos um certificado autoassinado com openssl, só para validação local, e configuramos o Nginx para servir HTTPS na porta 443.

Redirecionamento: Todo acesso via HTTP (porta 80) é redirecionado automaticamente para HTTPS.

Validação: Subimos a stack de produção com docker compose -f docker-compose-prod.yml up -d e confirmamos que o redirecionamento e o acesso via HTTPS funcionavam como esperado.

8. Etapa 6 – Deploy Contínuo (GHCR)

Imagens publicadas: As imagens de produção do backend e do frontend, publicadas automaticamente após os testes passarem.

Tags: Cada imagem recebe as tags latest e o hash do commit (github.sha), o que permite rastrear exatamente qual código gerou aquela versão.

Permissões: Usamos permissões mínimas no workflow (contents: read e packages: write), seguindo o princípio do menor privilégio.

9. Validação Final

Comandos executados: docker compose up --build, docker compose -f docker-compose-prod.yml up -d --build, docker images, docker history, python manage.py test e npm test.